Object pooling

Reusing objects instead of allocating them

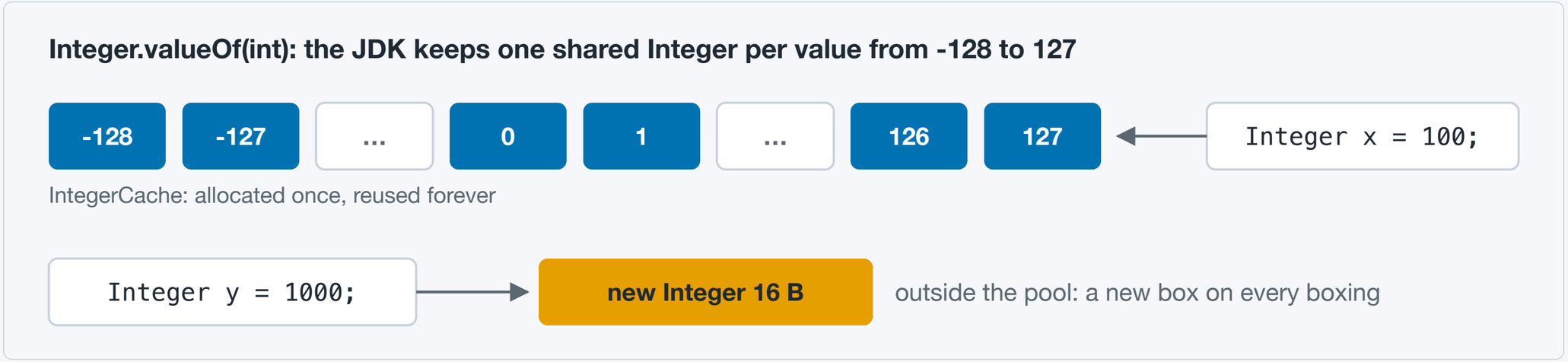
02

What is object pooling?

A pool keeps objects and hands them out again instead of allocating new ones.

- Callers return an object to the pool when they are done.
- **Worth it** for costly or limited resources: threads, connections, direct buffers.
- **Rarely worth it** for small, short-lived objects:
  - young-generation allocation is cheap;
  - pooled objects live long and need locking.

The JDK’s own pool: the `Integer` cache



- Autoboxing calls `Integer.valueOf(int)`.
- -128 to 127 come from the cache; any other value is a new 16 B object.

References

- `Integer.valueOf(int)` (Java SE 25 API): the -128 to 127 cache
- JLS SE 25, 5.1.7: Boxing Conversion
- The Java Tutorials: Thread Pools
- Java Garbage Collection Basics: why short-lived objects are cheap